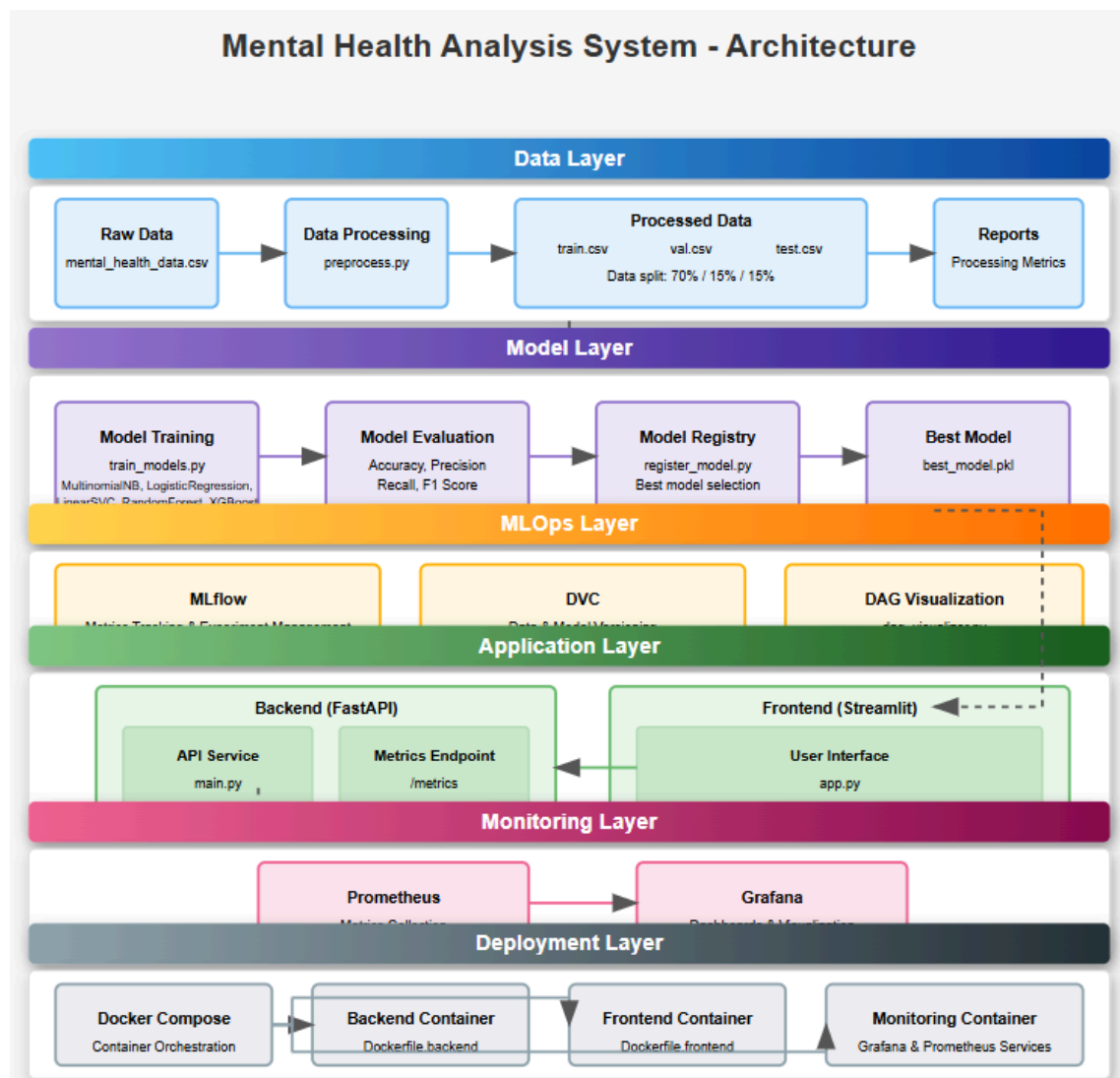


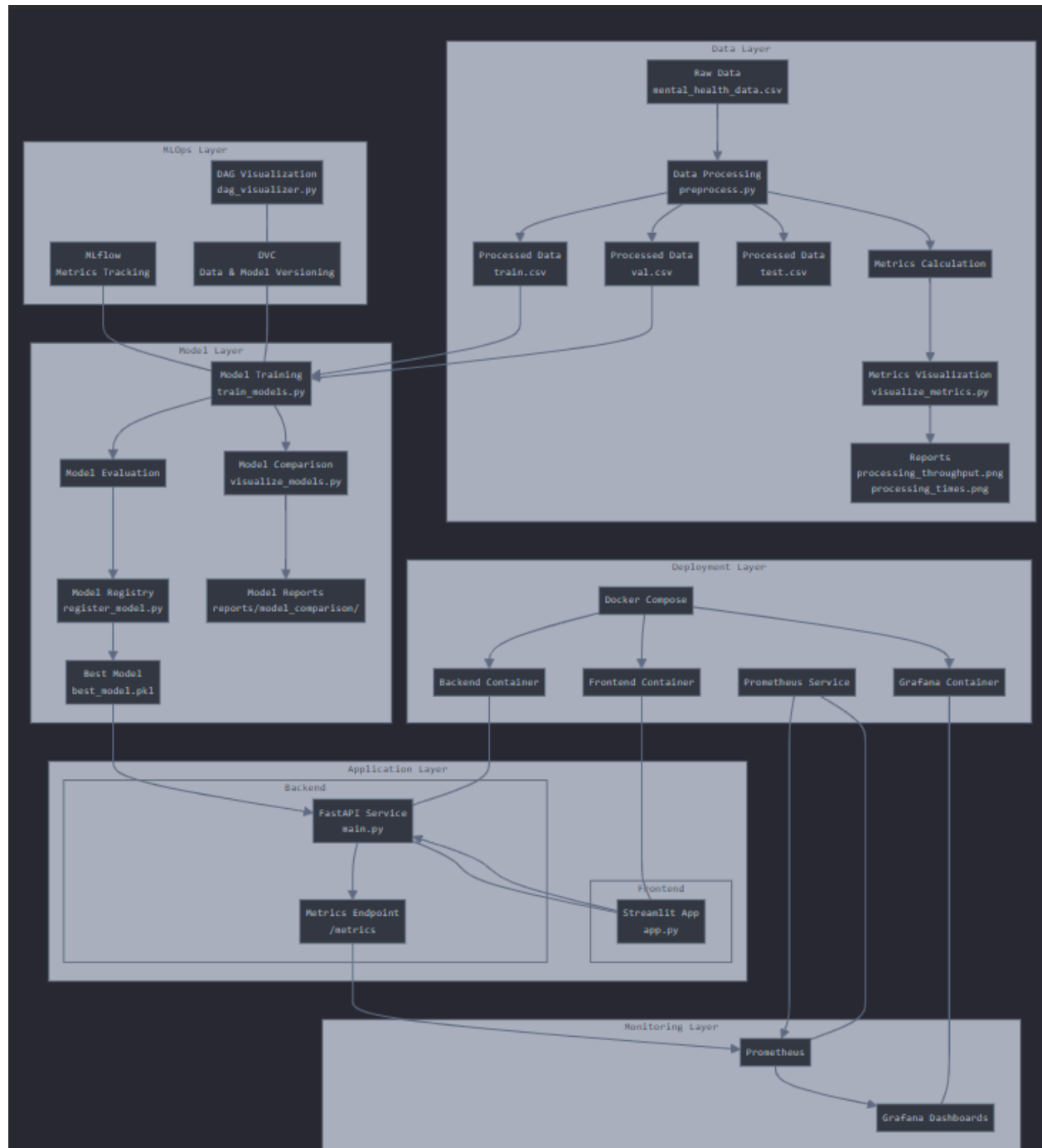
Name: Ananki Banerjee
Roll No.: DA24S017

Mental Health Analyzer - Documentation

High Level Design

Architecture Diagram





Overview

The Mental Health Analysis System is designed to analyze text inputs (journal entries) and classify them into mental health categories. The system is built using a machine learning approach, with a comprehensive pipeline covering data processing, model training, deployment, and monitoring.

Design Goals

1. **Accuracy:** Provide reliable mental health categorization of text
2. **Scalability:** Support increasing amounts of data and user requests
3. **Maintainability:** Ensure system components are decoupled and easily modifiable
4. **Observability:** Monitor system performance and model metrics
5. **User Experience:** Provide a simple, intuitive interface for users

System Components

Data Processing Subsystem

The data processing subsystem handles the transformation of raw mental health statement data into a format suitable for machine learning models.

Design Choices:

- **Data Splitting (70/15/15):** Provides sufficient training data while reserving adequate portions for validation and testing
- **Metric Calculation:** Records processing throughput and timing for performance optimization
- **Visualization:** Generates visual reports to facilitate data processing analysis

Model Training Subsystem

The model training subsystem is responsible for training multiple machine learning models and selecting the best performer.

Design Choices:

- **Multiple Model Approach:** Trains various models (Naive Bayes, Logistic Regression, SVM, RandomForest, XGBoost) to identify the best performer for this specific classification task
- **Text Vectorization Options:** Tests both TF-IDF and CountVectorizer approaches to identify optimal text representation
- **Metrics Focus:** Evaluates models based on accuracy, precision, recall, and other relevant metrics
- **Model Persistence:** Saves trained models for future use and comparisons

MLOps Integration

The MLOps integration ensures reproducibility, versioning, and tracking of experiments.

Design Choices:

- **MLflow:** Tracks model hyperparameters and performance metrics for experiment comparison
- **DVC:** Versions data and models to ensure reproducibility
- **DAG Visualization:** Provides clear visualization of pipeline dependencies
- **Model Registry:** Formalizes model selection process

Application Subsystem

The application subsystem provides the user interface and API services.

Design Choices:

- **Backend/Frontend Separation:** Decouples UI from processing logic
- **FastAPI Implementation:** Provides high-performance, async-capable API framework with automatic documentation
- **Streamlit Frontend:** Offers rapid UI development with Python-native components
- **RESTful Architecture:** Ensures standardized communication between components

Monitoring Subsystem

The monitoring subsystem tracks system performance and health metrics.

Design Choices:

- **Prometheus Integration:** Industry-standard metrics collection with dimensional data model
- **Custom Metrics:** Application-specific metrics to track usage patterns and model performance
- **Grafana Dashboards:** Visual representation of metrics for easy monitoring
- **15s Scrape Interval:** Balance between real-time monitoring and system overhead

Deployment Infrastructure

The deployment infrastructure containerises the application components for consistent deployment.

Design Choices:

- **Docker Containerization:** Ensures consistent environment across development and production
- **Component Isolation:** Separates frontend, backend, and monitoring services

- **Docker Compose Orchestration:** Simplifies multi-container deployment and configuration
- **Network Isolation:** Dedicated network for inter-service communication

Data Flow

1. **Data Ingestion and Processing:**
 - Raw data is processed through preprocessing scripts
 - Processed data is split and saved for training purposes
2. **Model Development:**
 - Multiple models are trained on processed data
 - Models are evaluated and compared
 - Best model is selected and registered
3. **Inference Flow:**
 - User enters text through Streamlit UI
 - Text is sent to FastAPI backend
 - Backend processes text using best model
 - Classification result is returned to UI and displayed
 - Metrics are updated for monitoring
4. **Monitoring Flow:**
 - Backend exposes metrics endpoint
 - Prometheus scrapes metrics at regular intervals
 - Grafana visualizes metrics in dashboards

Future Enhancements

1. **Model Retraining:** Automated retraining pipeline based on performance drift
2. **User Feedback Loop:** Mechanism to collect user feedback on predictions
3. **Advanced NLP:** Incorporation of more sophisticated language models
4. **High Availability:** Redundancy and failover mechanisms

Conclusion

The Mental Health Analyzer is designed as a modular, monitored application that leverages machine learning to provide mental health categorization. The architecture emphasizes clear separation of concerns, comprehensive monitoring, and reproducible ML workflows.

Low Level Design

API Endpoint Specifications

The Mental Health Analyzer API exposes four primary endpoints:

1. GET / - Root endpoint
 - Purpose: Health check / welcome message
 - Response: JSON {"message": "Welcome to Mental Health Analyzer API"}
 - Metrics tracked: Request count, total API calls
2. POST /predict - Mental health classification endpoint
 - Purpose: Analyzes text for mental health conditions
 - Request body: JSON with text field
 - Response: JSON with category and confidence fields
 - Processing: Feeds text to ML model to classify mental health condition
 - Metrics tracked: Request count, prediction latency, category distribution
3. GET /metrics - Prometheus metrics endpoint
 - Purpose: Exposes all API and system metrics in Prometheus format
 - Response: Plain text in Prometheus exposition format
 - Metrics tracked: Request count
4. GET /health - Service health check endpoint
 - Purpose: Service health monitoring
 - Response: JSON {"status": "healthy"}
 - Metrics tracked: Request count

Data Models

The API defines two primary Pydantic models:

1. TextInput
 - Fields:
 - text (string): The input text to analyze
2. PredictionOutput
 - Fields:
 - category (string): Predicted mental health category
 - confidence (float): Confidence score for the prediction

The API supports seven mental health categories:

- Normal
- Depression
- Anxiety
- Bipolar
- Personality disorder
- Stress
- Suicidal

Metrics Implementation

API Metrics

- `mental_health_api_requests_total` (Counter) - Total API requests by endpoint and method
- `mental_health_api_calls_total` (Counter) - Total number of API calls
- `mental_health_prediction_latency_seconds` (Histogram) - Time taken for prediction processing
- `mental_health_category_count` (Gauge) - Count of predictions by category
- `mental_health_category_distribution_percent` (Gauge) - Distribution percentage of predictions by category

System Metrics

- `system_cpu_usage_percent` (Gauge) - CPU utilization percentage
- `system_memory_usage_bytes` (Gauge) - Memory usage in bytes
- `system_disk_free_percent` (Gauge) - Disk free space percentage
- `system_disk_read_bytes_per_sec` (Gauge) - Disk read rate
- `system_disk_write_bytes_per_sec` (Gauge) - Disk write rate
- `system_network_io_sent_bytes_per_sec` (Gauge) - Network IO sent rate
- `system_network_io_received_bytes_per_sec` (Gauge) - Network IO received rate

Error Handling

The API implements several error handling mechanisms:

1. **Model Loading**
 - Attempts to load the model at startup
 - Logs errors if model loading fails
 - Raises an exception to prevent application startup with invalid model
2. **Prediction Errors**
 - Catches exceptions during prediction processing

- Logs detailed error information
 - Returns HTTP 500 with generic error message to user
 - Avoids exposing internal error details in responses
3. **Metrics Collection**
 - Background thread for metrics updates is wrapped in try-except
 - Errors in metrics collection are logged but don't affect API functionality
 - Automatic retry with increased delay after failures
 4. **Logging**
 - Comprehensive logging with multiple handlers (file and console)
 - Structured log format with timestamps and log levels
 - Different log levels for different event types

Configuration Parameters

The application uses the following configuration parameters:

1. **MODEL_PATH** (Environment Variable)
 - Default: "models/best_model.pkl"
 - Purpose: Path to the trained machine learning model file
2. **Logging Configuration**
 - Log level: INFO
 - Log format: Timestamp, logger name, level, message
 - Log destinations:
 - File: logs/api.log
 - Console output
3. **Metrics Configuration**
 - System metrics update interval: 15 seconds
 - Retry interval after error: 60 seconds
4. **Prometheus Metrics**
 - Metrics are automatically generated and exposed on the /metrics endpoint
 - Custom labels for detailed categorization

Deployment Configuration

1. Backend Dockerfile

```
FROM python:3.9-slim

# Set working directory
WORKDIR /app
```

```
# Copy requirements and install early for better caching
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application files
COPY models/ /app/models/
COPY main.py /app/

# Create logs directory for mounted logging
RUN mkdir -p /app/logs

# Set environment variables
ENV MODEL_PATH=/app/models/best_model.pkl
ENV PYTHONUNBUFFERED=1

# Expose ports for FastAPI (8000) and metrics (served on same port)
EXPOSE 8000

# Default command to run FastAPI app using uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2. Frontend Dockerfile

```
FROM python:3.9-slim

WORKDIR /app

# Install required dependencies
COPY frontend-requirements.txt .
RUN pip install --no-cache-dir -r frontend-requirements.txt

# Copy application code
COPY app.py /app/

# Create logs directory
RUN mkdir -p /app/logs

# Expose port
EXPOSE 8501
```

```
# Command to run the Streamlit app
CMD ["streamlit", "run", "app.py", "--server.port=8501",
"--server.address=0.0.0.0"]
```

3. Docker Compose Configuration

```
version: '3.8'

services:
  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    container_name: mental_health_backend
    ports:
      - "8000:8000"
    volumes:
      - ./logs:/app/logs
    networks:
      - mental_health_network
    restart: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    container_name: mental_health_frontend
    ports:
      - "8501:8501"
    volumes:
      - ./logs:/app/logs
    networks:
      - mental_health_network
    depends_on:
      - backend
    restart: always
```

```

grafana:
  image: grafana/grafana:latest
  container_name: mental_health_grafana
  ports:
    - "3000:3000"
  volumes:
    - grafana_data:/var/lib/grafana
  environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
    - GF_USERS_ALLOW_SIGN_UP=false
  networks:
    - mental_health_network
  # depends_on:
  #   - prometheus
  restart: always

networks:
  mental_health_network:
    driver: bridge

volumes:
  prometheus_data:
  grafana_data:

```

4. Prometheus Configuration

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'backend'
    static_configs:
      - targets: ['localhost:8000']
    metrics_path: /metrics

```

Test Plan and Test Cases

1. Test Strategy

1.1 Test Levels

- **Unit Testing:** Individual components and functions
- **Integration Testing:** Interaction between components
- **System Testing:** Complete system functionality
- **Performance Testing:** System performance under load
- **User Acceptance Testing:** End-user verification

1.2 Test Environments

- **Development:** Local environment for development testing
- **Testing:** Docker-based environment for integration testing
- **Staging:** Production-like environment for system testing
- **Production:** Live environment for monitoring

2. Unit Test Cases

2.1 Data Processing Module

Test Case	Input	Expected Output	Preconditions
Verify preprocessing of valid text	Raw text input	Processed text with lowercase, no special characters, etc.	None
Verify handling of empty input	Empty string	ValueError exception	None
Verify handling of non-text input	Numeric data	TypeError exception	None

Verify TF-IDF vectorization	Processed text	Vectorized representation	Vectorizer model loaded
Verify CountVectorizer transformation	Processed text	Vectorized representation	Vectorizer model loaded

2.2 Model Training Module

Test Case	Input	Expected Output	Preconditions
Verify model training with valid data	Training dataset	Trained model object	None
Verify model evaluation	Model and test data	Accuracy metrics	Trained model available
Verify model serialization	Model object	Serialized model file	Trained model available
Verify model deserialization	Model file path	Loaded model object	Model file exists
Verify prediction with loaded model	Vectorized text	Category prediction	Model loaded

2.3 API Module

Test Case	Input	Expected Output	Preconditions
Verify health check endpoint	GET request	Status 200 with healthy response	API running
Verify analyze endpoint with valid text	POST with valid text	Status 200 with prediction	API running, model loaded
Verify analyze endpoint with empty text	POST with empty text	Status 400 error	API running

Verify model info endpoint	GET request	Status 200 with model details	API running, model loaded
Verify metrics endpoint	GET request	Status 200 with Prometheus metrics	API running

2.4 Frontend Module

Test Case	Input	Expected Output	Preconditions
Verify UI rendering	App launch	UI components displayed	Streamlit running
Verify API connection	Mock API response	Successful data display	Streamlit running
Verify form validation	Submit empty form	Validation error message	Streamlit running
Verify form submission	Submit valid text	Results displayed	Streamlit running, API available
Verify error handling	API error	User-friendly error message	Streamlit running

3. Integration Test Cases

3.1 Data Processing to Model Training

Test Case	Input	Expected Output	Preconditions
Verify end-to-end preprocessing and training	Raw data	Trained model with expected accuracy	Raw data available
Verify metrics calculation during preprocessing	Raw data	Accurate throughput metrics	Raw data available
Verify visualization generation	Processing metrics	Generated visualization files	Metrics calculated

3.2 Model Training to API

Test Case	Input	Expected Output	Preconditions
Verify model loading in API	Best model path	Successful model initialization	Model file available
Verify prediction pipeline	Text input through API	Correct prediction with expected format	Model loaded in API
Verify metrics update after prediction	Multiple predictions	Updated Prometheus metrics	API running with metrics

3.3 API to Frontend

Test Case	Input	Expected Output	Preconditions
Verify frontend-backend communication	User input in UI	Successful API call and response	Both services running
Verify error propagation	Invalid input in UI	Proper error display	Both services running
Verify complete user flow	Journal entry submission	Category display with message	Complete system running

3.4 Monitoring Integration

Test Case	Input	Expected Output	Preconditions
Verify Prometheus scraping	API with metrics	Metrics available in Prometheus	API and Prometheus running
Verify Grafana dashboard	Prometheus data source	Visualized metrics in dashboard	Full monitoring stack
Verify custom metrics	Prediction activity	Updated custom metrics	Full system running

4. Test Data Requirements

4.1 Unit Testing Data

- Sample text inputs for each expected category
- Edge case inputs (empty, very short, very long)
- Invalid inputs for negative testing

4.2 Integration Testing Data

- Complete dataset samples
- Representative journal entries

4.3 Performance Testing Data

- Large volume of varied text inputs
- Scripts to generate concurrent requests

5. Test Environment Setup

5.1 Local Development Environment

Setup virtual environment

```
python -m venv venv
```

```
source venv/bin/activate # or venv\Scripts\activate on Windows
```

Install dependencies

```
pip install -r requirements.txt
```

```
pip install -r test_requirements.txt
```

Run the tests for each component.